

```
INSERT INTO users (fullname, email)
VALUES
  ('Ali Yılmaz', 'ali@example.com'),
  ('Ayşe Demir', 'ayse@example.com'),
  ('Mehmet Kaya', 'mehmet@example.com');
```

1 • `SELECT * FROM kullanıcı_db.users;`

Result Grid			
Filter Rows:			
Edit:    Export/Import: 			
	id	fullname	email
▶	1	Ali Yılmaz	ali@example.com
	2	Ayşe Demir	ayse@example.com
	3	Mehmet Kaya	mehmet@example.com
*	NULL	NULL	NULL

7 • `SELECT id, fullname, email FROM users;`

Result Grid			
Filter Rows:			
Edit:    Export/Import: 			
	id	fullname	email
▶	1	Ali Yılmaz	ali@example.com
	2	Ayşe Demir	ayse@example.com
	3	Mehmet Kaya	mehmet@example.com
*	NULL	NULL	NULL

- `UPDATE users`
`SET email = 'ali.yeni@example.com'`
`WHERE id = 1;`

- `SELECT * FROM users WHERE id = 1;`

Result Grid		
Filter Rows:		
Edit:   		
	id	email
▶	1	ali.yeni@example.com
NULL	NULL	NULL

```

12
13 • DELETE FROM users
14 WHERE id = 3;
15 • SELECT * FROM users;

```

Result Grid			
		Filter Rows:	
Edit:			
	id	fullname	email
▶	1	Ali Yılmaz	ali.yeni@example.com
	2	Ayşe Demir	ayse@example.com
•	NULL	NULL	NULL

Result Grid				
		Filter Rows:		
Edit:				
	id	user_id	order_number	order_date
▶	1	1	SIP-1001	2026-09-17 00:16:03
	2	1	SIP-1002	2026-09-17 00:16:03
	3	2	SIP-1003	2026-09-17 00:16:03
•	NULL	NULL	NULL	NULL

```

2 • SELECT
3     u.fullname,
4     u.email,
5     o.order_number
6 FROM users AS u
7 INNER JOIN orders AS o
8     ON u.id = o.user_id;

```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	fullname	email	order_number
▶	Ali Yılmaz	ali.yeni@example.com	SIP-1001
	Ali Yılmaz	ali.yeni@example.com	SIP-1002
	Ayşe Demir	ayse@example.com	SIP-1003

a) Ekran Görüntüsü ve Ekran Kaydı

Neden önemli?

Telefonumuzda banka şifresi, kart numarası veya hesap bakiyesi gibi bilgiler olabilir. Kötü amaçlı bir uygulama ekranımızı kaydederse bu bilgiler başkasının eline geçebilir.

Android:

FLAG_SECURE kullanılır. Açıldığında ekran görüntüsü alınamaz ve ekran kaydı sırasında hassas ekran görünmez.

iOS:

Ekran kaydı UIScreen.capturedDidChangeNotification ile takip edilebilir. Kayıt başladığında hassas bilgiler gizlenebilir. Şifre alanlarında ise isSecureTextEntry kullanılır.

Günlük hayat örneği:

Birinin omzunun üzerinden banka ekranına bakmasını engellemek gibi düşünebiliriz. Sistem ekranın üzerine perde çekiyor.

🔑 **ŞİFRE: “EKRAN = PERDE”**

→ Ekran kaydı/görüntüsü varsa **perdeyi kapat**.

b) Overlay (Üst Katman) Saldırıları

Nasıl gerçekleşir?

Kötü amaçlı uygulama, diğer uygulamaların üzerinde görünme iznini kullanır. Gerçek uygulamanın üzerine sahte veya görünmez bir ekran koyar.

Sen bankanın gerçek butonuna bastığını düşünürsün ama aslında saldırganın koyduğu sahte butona basarsın.

Örnek:

Sen “Bakiye Görüntüle” butonuna basmak isterken, üstteki görünmez katman bunu “Para Gönder” butonu olarak algılayabilir.

Günlük hayat örneği:

Gerçek sözleşmenin üzerine şeffaf bir sahte kağıt koymak gibidir. Sen gerçek kağıdı imzaladığını sanırsın ama aslında sahte kağıdı imzalamış olursun.

🔑 **ŞİFRE: “OVERLAY = SAHTE PERDE”**

→ Gerçek ekranın **üstüne sahte ekran** gelir.

c) Root / Jailbreak

Neden risklidir?

Normalde telefondaki uygulamalar birbirinden ayrı tutulur. Buna **sandbox** denir.

Root veya Jailbreak yapıldığında bu korumalar büyük ölçüde aşılabılır ve sisteme daha yüksek yetkiler verilir.

Bu durumda kötü amaçlı bir uygulama, normalde erişemeyeceği dosyalara veya uygulama verilerine ulaşabilir.

Örnek:

Bankacılık uygulamasının dosyalarına veya bellekte tuttuğu bazı bilgilere erişilmeye çalışılabilir.

Günlük hayat örneği:

Normalde herkesin ayrı odasının ve kilidinin olduğu bir otel düşün. Root/Jailbreak yapmak, otelin ana anahtarını ele geçirmek gibidir.

🔑 ŞİFRE: “ROOT = ANAHTAR”

→ Root/Jailbreak = **ana anahtar**

→ Ana anahtarı alan kişi birçok kapıyı açabilir.

d) SQLite ve Şifreleme

Sorun nedir?

Normal SQLite veritabanında bilgiler şifrlenmeden tutulabilir. Dosya ele geçirilirse kullanıcı adı, e-posta veya işlem bilgileri okunabilir.

SQLCipher ne yapar?

SQLCipher, SQLite veritabanını **AES ile şifreler**. Böylece dosya saldırganın eline geçse bile doğru anahtar olmadan verileri okuyamaz.

Günlük hayat örneği:

Normal SQLite, günlüğünü açık şekilde masanın üzerine bırakmak gibidir.

SQLCipher ise günlüğünü kilitli bir kasaya koymak gibidir.

🔑 ŞİFRE: “SQLITE = DEFTER, SQLCIPHER = KASA”

→ SQLite = Açık defter

→ SQLCipher = Kilitli kasa

e) Access Token ve Refresh Token

İkisini karıştırmamak için şöyle düşün:

Access Token:

Günlük işlemler için kullanılan **geçici karttır**. Genellikle kısa süre geçerlidir.

Refresh Token:

Access Token'ın süresi bitince yeni bir Access Token almak için kullanılan **daha değerli anahtardır**.

Neden Access Token kısa süreli?

Çalınırsa saldırganın kullanım süresi sınırlı olsun diye.

Refresh Token neden daha iyi korunmalı?

Çünkü yeni Access Token alınmasını sağlar. Bu yüzden Android'de **Keystore**, iOS'ta **Keychain** gibi güvenli alanlarda saklanması tercih edilir.

Logout neden önemli?

Çıkış yaptığında Refresh Token sunucu tarafında da geçersiz hale getirilmelidir. Böylece eski token kullanılarak tekrar oturum açılması engellenir.

Günlük hayat örneği:

Access Token = Oda kartı

→ Kısa süre kullanılır.



Refresh Token = Resepsiyondaki kimlik/pasaport

→ Yeni oda kartı almanı sağlar.



ŞİFRE: “ACCESS = KART, REFRESH = YENİ KART”

→ Access Token = **Geçici giriş kartı**

→ Refresh Token = **Yeni kart alma anahtarı**

→ Logout = **Anahtarı iptal et**